SPRINT 11 · VISIÓN POR COMPUTADOR · S11.1

# Redes convolucionales (CNN)

TurisData recibe cientos de fotos de alojamientos cada semana y las clasifica una persona a mano. ¿Puede un modelo pre-clasificarlas?

# Una imagen es una matriz de números

- **Escala de grises**: tabla de píxeles (en el cuaderno, 8×8 con valores de 0 = negro a 16 = blanco: los dígitos de `sklearn`).
- **Color**: 3 canales (rojo, verde, azul): forma `(alto, ancho, canales)`. Un lote: `(n, alto, ancho, canales)`.

```
X = (imagenes / 16.0)[..., np.newaxis]            # Ejercicio 1: (n, 8, 8, 1) entre 0 y 1
X_ent, X_pru, y_ent, y_pru = train_test_split(X, etiquetas, test_size=0.25,
                                              random_state=SEMILLA, stratify=etiquetas)
```

Keras espera un **canal explícito**.

# La convolución: un filtro que se desliza

- Un **filtro** (*kernel*) es una mini-tabla, por ejemplo 3×3.
- Se coloca sobre una zona, se multiplica casilla a casilla, se suma y sale **un** número; se desliza y se repite.
- El resultado es un **mapa de rasgos**: dónde aparece ese patrón (un borde, un trazo).

```
def tam_salida(n, k, p=0, s=1):                    # Ejercicio 2
    return (n - k + 2 * p) // s + 1                # k: filtro, p: relleno, s: paso
```

Con 8 píxeles y filtro 3 sin relleno: 6; con `padding="same"`, 8.

# Pooling: quedarse con lo importante

- *Max pooling* 2×2: se queda con el **máximo** de cada bloque. Mapas la mitad de grandes, menos cálculo, tolerancia a pequeños desplazamientos.

```
def max_pooling(mapa, t=2):                                  # Ejercicio 3
    a = mapa.shape[0] // t
    return mapa.reshape(a, t, a, t).max(axis=(1, 3))
```

Pregunta del cuaderno: ¿por qué una CNN necesita muchos menos parámetros que un MLP en imágenes grandes? Porque **reutiliza el mismo filtro** en toda la imagen.

# Una CNN completa en Keras

`[Conv → ReLU → Pooling] × varias veces` → **Flatten** → `Dense` → salida. Las primeras capas detectan trazos simples; las siguientes, formas cada vez más complejas.

```
def construir_cnn(filtros=(16, 32)):                              # Ejercicio 4
    m = keras.Sequential([keras.layers.Input((8, 8, 1))])
    for f in filtros:
        m.add(keras.layers.Conv2D(f, 3, padding="same", activation="relu"))
        m.add(keras.layers.MaxPooling2D(2))
    m.add(keras.layers.Flatten())
    m.add(keras.layers.Dense(10, activation="softmax"))
    m.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    return m
```

# Mirar dentro de la red

- Los **filtros de la primera capa** son pequeñas plantillas de trazos.
- Los **mapas de activación** muestran dónde se activa cada filtro con un dígito concreto.
- Es la forma de responder a «¿qué ha aprendido la red?» sin tratarla como una caja negra.

# *Data augmentation*

> Si el alojamiento aparece más a la izquierda, más pequeño o ligeramente girado, **sigue siendo el mismo alojamiento**.

- Genera variaciones aleatorias del entrenamiento (**solo al entrenar**): desplazar, girar, hacer zoom.
- Reduce el sobreajuste con pocas imágenes.
- Mini-reto: `cnn_aug` = la misma CNN con las capas de `aumento` al principio, 40 épocas.
- ¿En qué casos sería un error aplicar un giro aleatorio? (pregunta de reflexión: pista, dígitos «6» y «9»).

# Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

**S11_01_cnn** — 65 min

Convolución, pooling, CNN y *data augmentation*

sprints/S11/notebooks/S11_01_cnn.ipynb